ohmjs / ohm

Code | Issues 40 | Pull requests 5 | Discussions | Actions | Projects | Wiki | Sec

main | ohm / doc / | Go to file t | Add file | ...

millergarym and **Gary Miller** Docker release gha (#588) f9c53f3 · 2 weeks ago

Name	Name	Last commit date
..		
design	docs: use absolute links for images	5 months ago
images	doc: Add the logos!	8 months ago
releases	doc: change v18 migration guide ti...	last month
README.md	upgrade to ESLint 9	6 months ago
api-reference.md	Improve Instantiating Grammars d...	6 months ago
contributor-guide.md	doc: Fix broken link in readme, upd...	8 months ago
docker.md	Docker release gha (#588)	2 weeks ago
errors.md	Fix some typos in documentation (...	8 months ago
extras.md	upgrade to ESLint 9	6 months ago
incremental-semantics.md	doc: Fix minor grammatical issues i...	2 months ago
indentation-sensitive.md	doc: Update documentation on Ind...	3 years ago
patterns-and-pitfalls.md	Fix typo in patterns-and-pitfalls.md...	2 years ago
philosophy.md	Add `yarn format` script, and use it t...	5 years ago
publishing-grammars.md	Fix some typos in documentation (...	8 months ago
quick-reference.md	docs: use absolute links for images	5 months ago
syntax-reference.md	doc: Fix formatting in syntax refere...	8 months ago
typescript.md	doc: Fix generateBundles comman...	8 months ago
used-by.md	doc: fix typo in markdown	8 months ago

README.md

Ohm Documentation

- Ohm/JS API Reference
- Ohm Syntax Reference
- Learn more about the Ohm philosophy
- See Patterns and Pitfalls for some common Ohm patterns and solutions to frequently-encountered difficulties.

Examples

Here are some quick samples of what it's like to work with Ohm. For more in-depth examples, see the examples directory, especially the math example which is extensively commented.

Matching Strings

Instantiate a grammar from a string using `ohm.grammar()`, and check inputs using the grammar's `match()` method:

```
const ohm = require('ohm-js');
const g = ohm.grammar(`
  Laugh {
    laugh = lol | "lmao"
    lol = "l" "o"+ "l"
  }`);
assert(g.match('lol').succeeded());
assert(!g.match('lmao').failed());
assert(g.match('looooool').succeeded());
```

Implementing Semantics

You can use *operations* and *attributes* to analyze and extract values from parsed data. For example, take the following grammar in `arithmetic.ohm`:

```
Arithmetic {
  Exp
    = AddExp

  AddExp
    = AddExp "+" PriExp  -- plus
    | AddExp "-" PriExp  -- minus
    | PriExp

  PriExp
```

```
    = "(" Exp ")"  -- paren
    | number

  number
    = digit+
}
```

We can create an operation named 'eval' to evaluate arithmetic expressions that match the grammar:

```
// Instantiate the grammar.
const fs = require('fs');
const g = ohm.grammar(fs.readFileSync('arithmetic.ohm'));

// Create an operation that evaluates the expression. An operation always belongs to a Semant
// which is a family of related operations and attributes for a particular grammar.
const semantics = g.createSemantics().addOperation('eval', {
  Exp(e) {
    return e.eval();
  },
  AddExp(e) {
    return e.eval();
  },
  AddExp_plus(left, op, right) {
    return left.eval() + right.eval();
  },
  AddExp_minus(left, op, right) {
    return left.eval() - right.eval();
  },
  PriExp(e) {
    return e.eval();
  },
  PriExp_paren(open, exp, close) {
    return exp.eval();
  },
  number(chars) {
    return parseInt(this.sourceString, 10);
  },
});
const match = g.match('1 + (2 - 3) + 4');
assert.equal(semantics(match).eval(), 4);
```

You can learn more about semantics in the API reference.